Class Activity 1 — System Calls in Practice

> ***Related Lecture**: Week 2 — OS Structures & Interfaces*
> ***Topic**: Using POSIX System Calls in Linux*
> ***Language**: C*
> ***Environment**: Linux (native, VM, or WSL) + Windows (optional)*

> ⚠️ ***IMPORTANT — READ EVERYTHING FIRST***
>
> ***Before you write a single line of code, read through this ENTIRE document from top to bottom.** Scan every section — the examples, the tasks, the deliverables, the folder structure, and the README template. Understand the full scope of what is expected **before** you start working. Students who skip ahead often miss requirements and waste time redoing work.*
>
> ⏱️ ***Estimated time**: Reading this document takes about **30–40 minutes**. Completing all tasks takes approximately **3.5–5 hours** depending on your experience with C and Linux.*
>
> ***Document structure:***
>
> 1. ***Objective** — What you'll learn*
> 2. ***Prerequisites** — What you need set up*
> 3. ***Background: System Calls Explained** — How `write()`, `open()`, `read()`, `close()` work, what file descriptors are, and introductory examples for both Linux and Windows*
> 4. ***Task Overview** — Summary of all tasks at a glance*
> 5. ***Warm-Up Examples** — Starter code you run first to get comfortable*
> 6. ***Tasks 1–4 (Required)** — Detailed instructions; library code is provided, you write the system call version; Task 3 covers `strace` analysis; Task 4 explores OS structure*
> 7. ***Optional Bonus** — Windows API version*
> 8. ***Deliverables & Submission** — Folder structure, README template, git push*
> 9. ***Grading Criteria** — How your work will be evaluated*

Quick Navigation

Section	Jump To
Objective	▶ Objective
Prerequisites	▶ Prerequisites
Background: System Calls	▶ Background
Warm-Up Examples	▶ Warm-Up Examples
Task Overview	▶ Task Overview
Task 1: File Creator & Reader	▶ Task 1
Task 2: Directory Listing	▶ Task 2
Task 3: strace Analysis	▶ Task 3
Task 4: OS Structure	▶ Task 4

Optional Bonus: Windows	▶ Windows Bonus
Deliverables & Submission	▶ Deliverables
Grading Criteria	▶ Grading
References & Tips	▶ References · ▶ Tips

Objective

In lecture 2, you learned that **system calls** are the interface between user programs and the operating system kernel. In this activity, you will **experience the difference firsthand** by:

1. Running provided example programs that use system calls directly.
2. Studying complete C library programs (provided for you).
3. Writing your own equivalent programs using **POSIX system calls** that produce **identical output**.

This will help you understand what happens "under the hood" when you use familiar library functions like `printf()` and `fopen()`.

Prerequisites

- Basic C programming knowledge
- A Linux environment (Ubuntu VM, WSL, or native Linux)
- GCC compiler installed (`gcc --version` to verify)

Quick Setup (if using WSL on Windows)

```
# Install WSL if not already present (run in PowerShell as Admin)
wsl --install

# Inside WSL, install build tools
sudo apt update
sudo apt install build-essential -y
```

Background: System Calls Explained

Before jumping into the tasks, let's understand the key system call functions you'll be using.

What is a File Descriptor?

In Linux, **everything is a file** — including your terminal screen. When a program runs, the OS automatically opens three "files" for it:

File Descriptor	Name	What it represents
`0`	`STDIN_FILENO`	Standard input (keyboard)
`1`	`STDOUT_FILENO`	Standard output (terminal screen)
`2`	`STDERR_FILENO`	Standard error (terminal screen, for errors)

When you open a new file with `open()`, the OS assigns it the next available file descriptor (usually `3`, then `4`, etc.).

The Core POSIX System Calls

These are the four system calls you will use throughout this activity:

`write()` — Write data to a file descriptor

```
#include <unistd.h>

ssize_t write(int fd, const void *buf, size_t count);
```

Parameter	Meaning
`fd`	File descriptor to write to (`1` = terminal, or a file descriptor from `open()`)
`buf`	Pointer to the data you want to write
`count`	Number of bytes to write
Returns	Number of bytes actually written, or `-1` on error

`open()` — Open or create a file

```
#include <fcntl.h>

int open(const char *pathname, int flags, mode_t mode);
```

Parameter	Meaning
`pathname`	Path to the file (e.g., `"output.txt"`)
`flags`	How to open: `O_RDONLY` (read), `O_WRONLY` (write), `O_RDWR` (both), combined with `O_CREAT` (create if missing), `O_TRUNC` (erase existing content) using `|`
`mode`	Permissions if creating a new file (e.g., `0644` = owner read/write, others read-only)
Returns	A file descriptor (integer ≥ 0), or `-1` on error

`read()` — Read data from a file descriptor

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
```

Parameter	Meaning
`fd`	File descriptor to read from
`buf`	Buffer to store the data read

`count`	Maximum number of bytes to read
Returns	Number of bytes actually read (`0` = end of file), or `-1` on error

`close()` — Close a file descriptor

```
#include <unistd.h>

int close(int fd);
```

Parameter	Meaning
`fd`	File descriptor to close
Returns	`0` on success, `-1` on error

Windows Equivalent: `WriteFile()`

On Windows, the equivalent of `write()` is `WriteFile()`:

```
#include <windows.h>

BOOL WriteFile(HANDLE hFile, LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite,
               LPDWORD lpNumberOfBytesWritten, LPOVERLAPPED lpOverlapped);
```

Parameter	Meaning
`hFile`	Handle to the file or output device (use `GetStdHandle(STD_OUTPUT_HANDLE)` for terminal)
`lpBuffer`	Pointer to the data to write
`nNumberOfBytesToWrite`	Number of bytes to write
`lpNumberOfBytesWritten`	Pointer to a variable that receives the count of bytes written
`lpOverlapped`	Set to `NULL` for simple synchronous writes

Warm-Up Examples

Run these examples first to make sure your environment works and to see system calls in action.

Example 1: Hello with Linux System Call

This program prints a message to the terminal using the `write()` system call instead of `printf()`.

```
/* hello_syscall.c */
#include <unistd.h>

int main() {
    const char *message = "Hello, System Call!\n";
```

```
    write(1, message, 19);  // fd 1 = stdout, 19 = number of characters in the message
    return 0;
}
```

How it works:

- `write(1, message, 19)` tells the kernel: "write 19 bytes from `message` to file descriptor `1` (the terminal)."
- There is no formatting, no buffering — this goes directly to the kernel, which puts it on your screen.

Compile & run on Linux:

```
gcc -o hello_syscall hello_syscall.c
./hello_syscall
```

Expected output:

```
Hello, System Call!
```

Example 2: Hello with Windows API

This is the Windows equivalent — it uses `WriteFile()` to print to the console.

```
/* hello_winapi.c */
/* Compile with MinGW: gcc -o hello_winapi hello_winapi.c        */
/* Or with MSVC:       cl hello_winapi.c                          */

#include <windows.h>
#include <string.h>

int main() {
    const char *message = "Hello, Windows API!\n";
    DWORD bytesWritten;

    // Write to the console (STD_OUTPUT_HANDLE is stdout)
    WriteFile(GetStdHandle(STD_OUTPUT_HANDLE), message, strlen(message), &bytesWritten,
NULL);

    return 0;
}
```

How it works:

- `GetStdHandle(STD_OUTPUT_HANDLE)` gets the handle to the console (similar to file descriptor `1` in Linux).
- `WriteFile(...)` writes the bytes directly to that handle.
- `&bytesWritten` receives how many bytes were actually written.

Compile & run on Windows (CMD or PowerShell):

```
gcc -o hello_winapi hello_winapi.c
```

```
hello_winapi
```

Expected output:

```
Hello, Windows API!
```

> 📸 ***For the Windows example, take a screenshot of the output in your CMD/PowerShell or VS Code terminal.***

Example 3: File Copy with System Calls (`copyfilesyscall.c`)

This program copies the content of `result.txt` to `copyresult.txt` using only system calls.

First, create a test file:

```
echo "This is the content of result.txt" > result.txt
```

```c
/* copyfilesyscall.c */
#include <fcntl.h>
#include <unistd.h>

int main() {
    char buffer[1024];
    ssize_t bytesRead;

    // Open source file for reading
    int src = open("result.txt", O_RDONLY);
    if (src < 0) {
        const char *err = "Error: cannot open result.txt\n";
        write(2, err, 30);  // fd 2 = stderr
        return 1;
    }

    // Create/open destination file for writing (create if not exists, truncate if exists)
    int dst = open("copyresult.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dst < 0) {
        const char *err = "Error: cannot create copyresult.txt\n";
        write(2, err, 36);
        close(src);
        return 1;
    }

    // Read from source and write to destination, chunk by chunk
    while ((bytesRead = read(src, buffer, sizeof(buffer))) > 0) {
        write(dst, buffer, bytesRead);
    }

    // Close both files
    close(src);
    close(dst);
```

```
    const char *msg = "File copied successfully!\n";
    write(1, msg, 25);  // Print confirmation to terminal
    return 0;
}
```

How it works step by step:

1. `open("result.txt", O_RDONLY)` — opens the source file for reading only. Returns a file descriptor (e.g., `3`).
2. `open("copyresult.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)` — creates/opens the destination file for writing. `O_CREAT` creates it if missing, `O_TRUNC` erases any existing content. `0644` sets permissions.
3. `read(src, buffer, 1024)` — reads up to 1024 bytes from the source file into `buffer`. Returns the number of bytes read (or `0` at end of file).
4. `write(dst, buffer, bytesRead)` — writes exactly `bytesRead` bytes from `buffer` to the destination file.
5. The `while` loop repeats until `read()` returns `0` (end of file).
6. `close()` releases each file descriptor.

Compile & run:

```
gcc -o copyfilesyscall copyfilesyscall.c
./copyfilesyscall
cat copyresult.txt
```

Expected output:

```
File copied successfully!
```

And `copyresult.txt` should contain the same content as `result.txt`.

Task Overview

Now that you've seen the examples, here is what you need to do. **The library version (Version A) is provided for you. Your job is to write the system call version (Version B).**

Task	What You'll Build	You Are Given	You Write	Key System Calls
1. File Creator & Reader	Create a file, write text, then read it back	`file_creator_lib.c` + `file_reader_lib.c` (complete)	`file_creator_sys.c` + `file_reader_sys.c`	`open`, `write`, `read`, `close`
2. Directory Listing	List files in current directory with sizes	`dir_list_lib.c` (complete)	`dir_list_sys.c`	`opendir`, `readdir`, `stat`, `write`, `close`
3. strace Analysis	Trace & compare system calls of both versions	Tasks 1–2 programs	Annotated screenshots + written analysis	`strace`

4. OS Structure	Explore how Linux exposes kernel info via `/proc`	Commands provided	Screenshots + written analysis	`/proc`, `uname`, `lsmod`
Bonus (Optional)	Redo Task 1 using Windows API	—	`file_creator_win.c`	`CreateFile`, `WriteFile`, `CloseHandle`

Both versions must produce identical output.

Required Tasks

Task 1: File Creator & Reader

Part A — File Creator

Write a program that:

- Creates a new file called `output.txt`
- Writes the text `"Hello from Operating Systems class!\n"` into the file
- Closes the file
- Prints `"File created successfully!\n"` to the terminal

Version A — Library Functions (PROVIDED)

This code is complete. Compile it, run it, and verify it works. Then write your Version B to match its behavior.

```c
/* file_creator_lib.c */
#include <stdio.h>

int main() {
    FILE *fp = fopen("output.txt", "w");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    fprintf(fp, "Hello from Operating Systems class!\n");
    fclose(fp);

    printf("File created successfully!\n");
    return 0;
}
```

Compile & run:

```
gcc -o file_creator_lib file_creator_lib.c
./file_creator_lib
cat output.txt
```

Version B — Using POSIX System Calls (YOU WRITE THIS)

Rewrite the program above using system calls: `open()` , `write()` , `close()` .

Hints:

- Use `open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644)` to create the file.
- Use `write(fd, text, length)` to write to the file.
- Use `write(1, msg, length)` to print the confirmation to the terminal (fd `1` = stdout).
- Use `close(fd)` to close the file.

```c
/* file_creator_sys.c */
#include <fcntl.h>    // open(), O_WRONLY, O_CREAT, O_TRUNC
#include <unistd.h>   // write(), close()
#include <string.h>   // strlen()

int main() {
    // YOUR CODE HERE
    // 1. Open/create "output.txt" using open()
    // 2. Write "Hello from Operating Systems class!\n" using write()
    // 3. Close the file using close()
    // 4. Print "File created successfully!\n" to the terminal using write()
    return 0;
}
```

Compile & run:

```
gcc -o file_creator_sys file_creator_sys.c
./file_creator_sys
cat output.txt
```

Both versions must produce the same output in the terminal and the same content in `output.txt` .

Questions (answer in your README)

1. What flags did you pass to `open()` ? What does each flag mean?
2. What is `0644` in the permission argument? What does each digit represent?
3. What does `fopen("output.txt", "w")` do internally that you had to do manually with `open()` ?

Part B — File Reader & Display

Now write a program that:

- Opens the file `output.txt` (created in Part A)
- Reads its contents
- Displays the contents to the terminal
- Closes the file

Version A — Library Functions (PROVIDED)

This code is complete. Compile it, run it, then write your Version B.

```c
/* file_reader_lib.c */
#include <stdio.h>

int main() {
    FILE *fp = fopen("output.txt", "r");
    if (fp == NULL) {
        perror("Error opening file");
        return 1;
    }

    char buffer[256];
    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }

    fclose(fp);
    return 0;
}
```

Compile & run:

```
gcc -o file_reader_lib file_reader_lib.c
./file_reader_lib
```

Version B — Using POSIX System Calls (YOU WRITE THIS)

Rewrite using: `open()`, `read()`, `write()`, `close()`.

Hints:

- Use `open("output.txt", O_RDONLY)` to open the file for reading.
- Use `read(fd, buffer, sizeof(buffer))` to read data into a buffer. It returns the number of bytes read.
- Use `write(1, buffer, bytesRead)` to print the data to the terminal.
- Loop until `read()` returns `0` (end of file).

```c
/* file_reader_sys.c */
#include <fcntl.h>
#include <unistd.h>

int main() {
    char buffer[256];
    // YOUR CODE HERE
    // 1. Open "output.txt" for reading using open()
    // 2. Read content into buffer using read() in a loop
    // 3. Write the content to the terminal (fd 1) using write()
    // 4. Close the file using close()
    return 0;
}
```

Both versions must display the same content on the terminal.

Questions (answer in your README)

1. What does `read()` return? How is this different from `fgets()` ?
2. Why do you need a loop when using `read()` ? When does it stop?

Task 2: Directory Listing & File Info

Write a program that:

- Lists all files in the current directory
- For each file, prints its name and size in bytes

Version A — Library Functions (PROVIDED)

This code is complete. Compile it, run it, then write your Version B.

```c
/* dir_list_lib.c */
#include <stdio.h>
#include <dirent.h>
#include <sys/stat.h>

int main() {
    DIR *dir = opendir(".");
    if (dir == NULL) {
        perror("Error opening directory");
        return 1;
    }

    struct dirent *entry;
    struct stat fileStat;

    printf("%-30s %10s\n", "Filename", "Size (bytes)");
    printf("%-30s %10s\n", "------------------------------", "----------");

    while ((entry = readdir(dir)) != NULL) {
        if (stat(entry->d_name, &fileStat) == 0) {
            printf("%-30s %10ld\n", entry->d_name, fileStat.st_size);
        }
    }

    closedir(dir);
    return 0;
}
```

Compile & run:

```
gcc -o dir_list_lib dir_list_lib.c
./dir_list_lib
```

Version B — Using System Calls (YOU WRITE THIS)

Rewrite the output portion using `write()` instead of `printf()` . You may still use `opendir()` , `readdir()` , and `stat()` (these are thin wrappers over system calls).

Hints:

- Use `snprintf(buffer, sizeof(buffer), "%-30s %10ld\n", name, size)` to format text into a buffer, then `write(1, buffer, len)` to output it.
- Use `strlen()` to get the length of the formatted string.

```
/* dir_list_sys.c */
#include <fcntl.h>
#include <unistd.h>
#include <dirent.h>
#include <sys/stat.h>
#include <string.h>
#include <stdio.h>    // only for snprintf to format numbers into strings

int main() {
    char buffer[512];
    // YOUR CODE HERE
    // 1. Open current directory with opendir(".")
    // 2. Print header line using write()
    // 3. Loop through entries with readdir()
    // 4. For each entry, use stat() to get file size
    // 5. Format output into buffer with snprintf(), then write() to fd 1
    // 6. Close directory with closedir()
    return 0;
}
```

Both versions must display the same file listing.

Questions (answer in your README)

1. What struct does `readdir()` return? What fields does it contain?
2. What information does `stat()` provide beyond file size?
3. Why can't you just `write(1, some_number, ...)` directly — why do you need `snprintf()` first?

Task 3: Tracing System Calls with `strace`

Now that you have both library and system call versions of your programs, let's see **exactly** what the operating system does when each version runs. The `strace` tool intercepts and records every system call a program makes.

What is `strace`?

`strace` is a Linux diagnostic tool that shows you every system call a program makes in real time. It reveals the "hidden" system calls that library functions generate behind the scenes.

```
# Basic usage
strace ./your_program

# Save output to a file (strace writes to stderr, so redirect with 2>)
strace ./your_program 2> strace_output.txt

# Show only specific system calls (e.g., file-related)
strace -e trace=open,openat,read,write,close ./your_program
```

```
# Count system calls (summary)
strace -c ./your_program
```

What to Do

Step 1 — Run `strace` on both versions of Task 1:

```
# Trace the library version
strace -e trace=openat,read,write,close ./file_creator_lib 2> strace_lib_task1.txt

# Trace the system call version
strace -e trace=openat,read,write,close ./file_creator_sys 2> strace_sys_task1.txt
```

Step 2 — Run `strace` on the file reader or directory listing programs:

Pick at least one more program pair and trace both versions:

```
# Example with the file reader (Task 1, Part B)
strace -e trace=openat,read,write,close ./file_reader_lib 2> strace_lib_reader.txt
strace -e trace=openat,read,write,close ./file_reader_sys 2> strace_sys_reader.txt
```

Step 3 — Get a system call summary:

```
# Summary for library version
strace -c ./file_creator_lib 2> strace_summary_lib.txt

# Summary for system call version
strace -c ./file_creator_sys 2> strace_summary_sys.txt
```

Step 4 — Capture and annotate screenshots:

1. Take a screenshot of the `strace` output for the **library version** of at least two programs (e.g., file creator and file reader).
2. Take a screenshot of the `strace` output for the **system call version** of the same programs.
3. Take a screenshot of the `strace -c` summary for both versions.
4. **Highlight or annotate** the key differences in your screenshots (use boxes, arrows, or colored highlights). You can use any tool: Paint, Snipping Tool markup, PowerPoint, or an image editor.

> 📸 ***Your screenshots must clearly highlight the differences*** *— do not just paste raw output. Circle or box the important lines and add short labels explaining what they mean.*

Example of what to highlight:

- In the library version: extra `brk()` or `mmap()` calls for buffering, `openat()` calls to load shared libraries
- In the syscall version: direct `openat()`, `write()`, `close()` calls with no extra overhead
- Differences in the number of `write()` calls (library may buffer multiple writes into one)

Questions (answer in your README)

1. How many system calls does the **library version** of Task 1 make compared to the **system call version**? (Use `strace -c` output)

2. What extra system calls do you see in the library version that are not in the system call version? What do they do? (e.g., `brk`, `mmap`, `access`, `fstat`)
3. When the library version calls `fprintf()`, how many actual `write()` system calls does `strace` show? Does it match what you expected? Why or why not?
4. Based on the `strace` output, explain in your own words: **What is the real difference between a library function and a system call?**

Task 4: Exploring OS Structure through `/proc`

In lecture, you learned that the OS has a layered structure: **hardware → kernel → system calls → user programs**. In this task, you will observe this structure in a running Linux system by exploring the `/proc` virtual filesystem — a window into the kernel's internal state.

What is `/proc`?

`/proc` is a **virtual filesystem** — it doesn't store anything on disk. Instead, the kernel generates its contents on the fly when you read them. Each file in `/proc` exposes real-time information about the system: CPU details, memory usage, running processes, loaded kernel modules, and more.

Part A — System Information

Run the following commands and capture the output:

```
# Kernel version and system info
uname -a

# CPU information
cat /proc/cpuinfo | head -20

# Memory information
cat /proc/meminfo | head -10

# Kernel version string
cat /proc/version

# System uptime (in seconds)
cat /proc/uptime
```

> 📸 ***Take a screenshot*** *showing the output of all five commands.*

Part B — Process Information

Every running process has a directory under `/proc/<PID>/`. You can inspect your own process using `/proc/self/`:

```
# Your shell's process status
cat /proc/self/status | head -20

# Memory map of the current process
cat /proc/self/maps | head -20

# Command line used to start the process
```

```
cat /proc/self/cmdline | tr '\0' ' '; echo

# List all running processes (traditional view)
ps aux | head -15
```

> 📸 ***Take a screenshot*** *showing the output of these commands.*

Part C — Kernel Modules

The Linux kernel is **modular** — functionality can be loaded and unloaded at runtime:

```
# List currently loaded kernel modules
lsmod | head -20

# Get details about a specific module (pick one from lsmod output)
modinfo <module_name>
```

> 📸 ***Take a screenshot*** *of* `lsmod` *output and one* `modinfo` *result.*

Part D — Draw the OS Layers

Based on what you found in Parts A–C, draw a **layered diagram** of your system's OS structure. Your diagram should include:

1. **Hardware layer** — Label with your actual CPU model (from `/proc/cpuinfo`) and memory size (from `/proc/meminfo`)
2. **Kernel layer** — Label with your kernel version (from `uname -a`) and list 2–3 loaded modules (from `lsmod`)
3. **System call interface** — Show where system calls like `open()`, `read()`, `write()` sit
4. **User space** — Show your shell process and the programs you ran in Tasks 1–2

You can draw this by hand (photo), in a drawing tool (draw.io, PowerPoint, Paint), or as ASCII art.

> 📸 ***Include your diagram*** *in your* `screenshots/` *folder as* `task4_os_layers_diagram.png` *(or* `.jpg`*).*

Questions (answer in your README)

1. What is `/proc`? Is it a real filesystem on disk? Where does its content come from?
2. What is the difference between a **monolithic kernel** and a **microkernel**? Based on the `lsmod` output, which type does Linux use?
3. Look at the output of `cat /proc/self/maps`. What different memory regions do you see? (e.g., heap, stack, shared libraries)
4. What does the kernel version string (from `uname -a`) tell you? Break down each part.
5. How does `/proc` demonstrate that the OS acts as an **intermediary** between user programs and hardware?

Optional Bonus: System Calls on Windows

> *This section is* ***optional*** *for students who want to explore system programming on Windows.*

Windows does not use POSIX. Instead, it uses the **Windows API (WinAPI)** for system-level operations. Rewrite **Task 1 (File Creator)** using WinAPI.

Version W — Using Windows API

```c
/* file_creator_win.c */
/* Compile with MinGW: gcc -o file_creator_win file_creator_win.c  */
/* Or with MSVC:       cl file_creator_win.c                       */

#include <windows.h>
#include <string.h>

int main() {
    const char *text = "Hello from Operating Systems class!\n";
    const char *confirm = "File created successfully!\n";
    DWORD bytesWritten;

    // Create/open the file
    HANDLE hFile = CreateFile(
        "output.txt",           // filename
        GENERIC_WRITE,          // access mode
        0,                      // no sharing
        NULL,                   // default security
        CREATE_ALWAYS,          // always create new (overwrite if exists)
        FILE_ATTRIBUTE_NORMAL,  // normal file
        NULL                    // no template
    );

    if (hFile == INVALID_HANDLE_VALUE) {
        const char *err = "Error creating file\n";
        WriteFile(GetStdHandle(STD_ERROR_HANDLE), err, strlen(err), &bytesWritten, NULL);
        return 1;
    }

    // Write text to the file
    WriteFile(hFile, text, strlen(text), &bytesWritten, NULL);

    // Close the file
    CloseHandle(hFile);

    // Print confirmation to console
    WriteFile(GetStdHandle(STD_OUTPUT_HANDLE), confirm, strlen(confirm), &bytesWritten,
NULL);

    return 0;
}
```

> 📸 ***Screenshot the output of running this on Windows (CMD, PowerShell, or VS Code terminal) and include it in your README.***

Cross-Platform Comparison Table

Operation	C Library	POSIX System Call (Linux)	Windows API

Create/open file	`fopen()`	`open()`	`CreateFile()`
Write to file	`fprintf()`	`write()`	`WriteFile()`
Read from file	`fread()`	`read()`	`ReadFile()`
Close file	`fclose()`	`close()`	`CloseHandle()`
Delete file	`remove()`	`unlink()`	`DeleteFile()`
Print to console	`printf()`	`write(1, ...)`	`WriteFile(GetStdHandle(...), ...)`

Bonus Questions (answer in your README)

1. Why does Windows use `HANDLE` instead of integer file descriptors?
2. What is the Windows equivalent of POSIX `fork()`? Why is it different?
3. Can you use POSIX calls on Windows? (Hint: look up WSL and Cygwin)

Deliverables & Submission

What to Submit

Everything goes in your **README.md** — there is no separate answers file. Your `README.md` must contain:

1. **Your name and student ID**
2. **Screenshots** of compiling and running each program (both versions side by side)
3. **Answers** to all questions listed under each task
4. **Source code** files (`.c` files in the task folders)

Submission Folder Structure

All class activities go inside your personal `os-se-<YourStudentID>/` repository, in a dedicated folder following the same convention as labs:

```
os-se-<YourStudentID>/
├── os-lab-<YourStudentID>/               # ← Your lab submissions (already exists)
│   ├── lab1/
│   ├── lab2/
│   └── ...
│
└── os-class-activities-<YourStudentID>/   # ← Your class activity submissions
    └── activity1/
        ├── README.md                     # ← Your report: screenshots + answers + reflection
        ├── screenshots/                  # Screenshot images
        │   ├── task1_creator_lib.png
        │   ├── task1_creator_sys.png
        │   ├── task1_reader_lib.png
        │   ├── task1_reader_sys.png
        │   ├── task2_lib.png
        │   ├── task2_sys.png
        │   ├── strace_*.png
        │   ├── task4_system_info.png
        │   ├── task4_os_layers_diagram.png
```

```
        │   └── ...
        ├── task1/
        │   ├── file_creator_lib.c         # Provided (copy from instructions)
        │   ├── file_creator_sys.c         # YOU WRITE THIS
        │   ├── file_reader_lib.c          # Provided (copy from instructions)
        │   ├── file_reader_sys.c          # YOU WRITE THIS
        │   └── file_creator_win.c         # (optional bonus)
        └── task2/
            ├── dir_list_lib.c             # Provided (copy from instructions)
            └── dir_list_sys.c             # YOU WRITE THIS
```

Setting Up Your Activity Folder

```
# Navigate to your existing submission repo
$ cd os-se-<YourStudentID>

# Create the class activities folder structure
$ mkdir -p os-class-activities-
<YourStudentID>/activity1/{task1,task2,task3_strace,task4_os_structure,screenshots}

# Start working
$ cd os-class-activities-<YourStudentID>/activity1
```

Pushing to Git

This uses the **same repository** you already created for your labs. Just commit and push the new folder:

```
# Make sure you are in the root of your repo
$ cd os-se-<YourStudentID>

# Stage your class activity files
$ git add os-class-activities-<YourStudentID>/

# Commit with a meaningful message
$ git commit -m "Add class activity 1 — System Calls in Practice"

# Push to your remote repository
$ git push origin main
```

> ***Reminder:*** *This is the same* `os-se-<YourStudentID>` *GitHub repository you use for lab submissions. Your labs and class activities live side by side in the same repo.*
> *Commit and push* ***regularly*** *as you work — do not wait until the last minute.*

README Template for Activity 1

Create a `README.md` inside your `activity1/` folder using this template:

```
# Class Activity 1 — System Calls in Practice

- **Student Name:** [Your Name Here]
```

- **Student ID:** [Your Student ID Here]
- **Date:** [Date of Submission]

Warm-Up: Hello System Call

Screenshot of running `hello_syscall.c` on Linux:

![Hello syscall](screenshots/hello_syscall.png)

Screenshot of running `hello_winapi.c` on Windows (CMD/PowerShell/VS Code):

![Hello WinAPI](screenshots/hello_winapi.png)

Screenshot of running `copyfilesyscall.c` on Linux:

![Copy file syscall](screenshots/copyfilesyscall.png)

Task 1: File Creator & Reader

Part A — File Creator

Describe your implementation: [What differences did you notice between the library version and the system call version?]

Version A — Library Functions (`file_creator_lib.c`):

<!-- Screenshot: gcc -o file_creator_lib file_creator_lib.c && ./file_creator_lib && cat output.txt -->
![Task 1A - Library](screenshots/task1_creator_lib.png)

Version B — POSIX System Calls (`file_creator_sys.c`):

<!-- Screenshot: gcc -o file_creator_sys file_creator_sys.c && ./file_creator_sys && cat output.txt -->
![Task 1A - Syscall](screenshots/task1_creator_sys.png)

Questions:

1. **What flags did you pass to `open()`? What does each flag mean?**

   > [Your answer]

2. **What is `0644`? What does each digit represent?**

   > [Your answer]

3. **What does `fopen("output.txt", "w")` do internally that you had to do manually?**

> [Your answer]

Part B — File Reader & Display

Describe your implementation: [Your notes]

Version A — Library Functions (`file_reader_lib.c`):

![Task 1B - Library](screenshots/task1_reader_lib.png)

Version B — POSIX System Calls (`file_reader_sys.c`):

![Task 1B - Syscall](screenshots/task1_reader_sys.png)

Questions:

1. **What does `read()` return? How is this different from `fgets()`?**

   > [Your answer]

2. **Why do you need a loop when using `read()`? When does it stop?**

   > [Your answer]

Task 2: Directory Listing & File Info

Describe your implementation: [Your notes]

Version A — Library Functions (`dir_list_lib.c`)

![Task 2 - Version A](screenshots/task2_lib.png)

Version B — System Calls (`dir_list_sys.c`)

![Task 2 - Version B](screenshots/task2_sys.png)

Questions

1. **What struct does `readdir()` return? What fields does it contain?**

   > [Your answer]

2. **What information does `stat()` provide beyond file size?**

   > [Your answer]

3. **Why can't you `write()` a number directly — why do you need `snprintf()` first?**

   > [Your answer]

Optional Bonus: Windows API (`file_creator_win.c`)

Screenshot of running on Windows:

![Task 1 - Windows](screenshots/task1_win.png)

Bonus Questions

1. **Why does Windows use `HANDLE` instead of integer file descriptors?**

   > [Your answer]

2. **What is the Windows equivalent of POSIX `fork()`? Why is it different?**

   > [Your answer]

3. **Can you use POSIX calls on Windows?**

   > [Your answer]

Task 3: strace Analysis

Describe what you observed: [What surprised you about the strace output? How many more system calls did the library version make?]

strace Output — Library Version (File Creator)

<!-- Screenshot: strace -e trace=openat,read,write,close ./file_creator_lib -->
<!-- IMPORTANT: Highlight/annotate the key system calls in your screenshot -->
![strace - Library version File Creator](screenshots/strace_lib_creator.png)

strace Output — System Call Version (File Creator)

<!-- Screenshot: strace -e trace=openat,read,write,close ./file_creator_sys -->
<!-- IMPORTANT: Highlight/annotate the key system calls in your screenshot -->
![strace - System call version File Creator](screenshots/strace_sys_creator.png)

strace Output — Library Version (File Reader or Dir Listing)

![strace - Library version](screenshots/strace_lib_reader.png)

strace Output — System Call Version (File Reader or Dir Listing)

![strace - System call version](screenshots/strace_sys_reader.png)

strace -c Summary Comparison

<!-- Screenshot of `strace -c` output for both versions -->

```
![strace summary - Library](screenshots/strace_summary_lib.png)
![strace summary - Syscall](screenshots/strace_summary_sys.png)

### Questions

1. **How many system calls does the library version make compared to the system call version?**

   > [Your answer — use the `strace -c` counts]

2. **What extra system calls appear in the library version? What do they do?**

   > [Your answer — mention `brk`, `mmap`, `fstat`, etc.]

3. **How many `write()` calls does `fprintf()` actually produce?**

   > [Your answer]

4. **In your own words, what is the real difference between a library function and a system call?**

   > [Your answer]

---

## Task 4: Exploring OS Structure

### System Information

> 📸 Screenshot of `uname -a`, `/proc/cpuinfo`, `/proc/meminfo`, `/proc/version`, `/proc/uptime`:

![System Info](screenshots/task4_system_info.png)

### Process Information

> 📸 Screenshot of `/proc/self/status`, `/proc/self/maps`, `ps aux`:

![Process Info](screenshots/task4_process_info.png)

### Kernel Modules

> 📸 Screenshot of `lsmod` and `modinfo`:

![Kernel Modules](screenshots/task4_modules.png)

### OS Layers Diagram

> 📸 Your diagram of the OS layers, labeled with real data from your system:

![OS Layers Diagram](screenshots/task4_os_layers_diagram.png)
```

```
### Questions

1. **What is `/proc`? Is it a real filesystem on disk?**

   > [Your answer]

2. **Monolithic kernel vs. microkernel — which type does Linux use?**

   > [Your answer]

3. **What memory regions do you see in `/proc/self/maps`?**

   > [Your answer]

4. **Break down the kernel version string from `uname -a`.**

   > [Your answer]

5. **How does `/proc` show that the OS is an intermediary between programs and hardware?**

   > [Your answer]

---

## Reflection

What did you learn from this activity? What was the most surprising difference between
library functions and system calls?

> [Write your reflection here]
```

> ***Tip:*** *Create a* `screenshots/` *folder inside* `activity1/` *and place all your screenshot images there. Use relative paths in the README as shown above.*

Grading Criteria

Criteria	Points
Warm-up examples run successfully (with screenshots)	5
Task 1A — File Creator syscall version compiles and matches output	10
Task 1B — File Reader syscall version compiles and matches output	10
Task 2 — Directory Listing syscall version compiles and matches output	10
Task 3 — strace screenshots with clear highlights and annotations	10
Task 3 — strace questions answered with evidence from output	10
Task 4 — OS structure screenshots and OS layers diagram	10
Task 4 — OS structure questions answered thoughtfully	10

All other questions (Tasks 1–2) answered thoughtfully in README	15
README is well-organized with clear screenshots	10
Bonus: Windows API version with screenshot	+10
Total	**100 (+10 bonus)**

Helpful References

- **POSIX System Calls**: `man 2 open`, `man 2 read`, `man 2 write`, `man 2 close`, `man 2 stat`
- **C Library Functions**: `man 3 fopen`, `man 3 fread`, `man 3 printf`
- **Linux man pages online**: https://man7.org/linux/man-pages/
- **Windows API File I/O**: https://learn.microsoft.com/en-us/windows/win32/fileio/file-management-functions

---

Tips

- Always check return values! System calls return `-1` on error; use `perror()` or write an error message to fd `2` (stderr).
- Use `strace ./your_program` on Linux to see the actual system calls your program makes — try it on both versions!
- The `strace` output for Version A will show you that library functions are just wrappers around system calls.
- Start with the warm-up examples. If those compile and run, you're ready for the tasks.